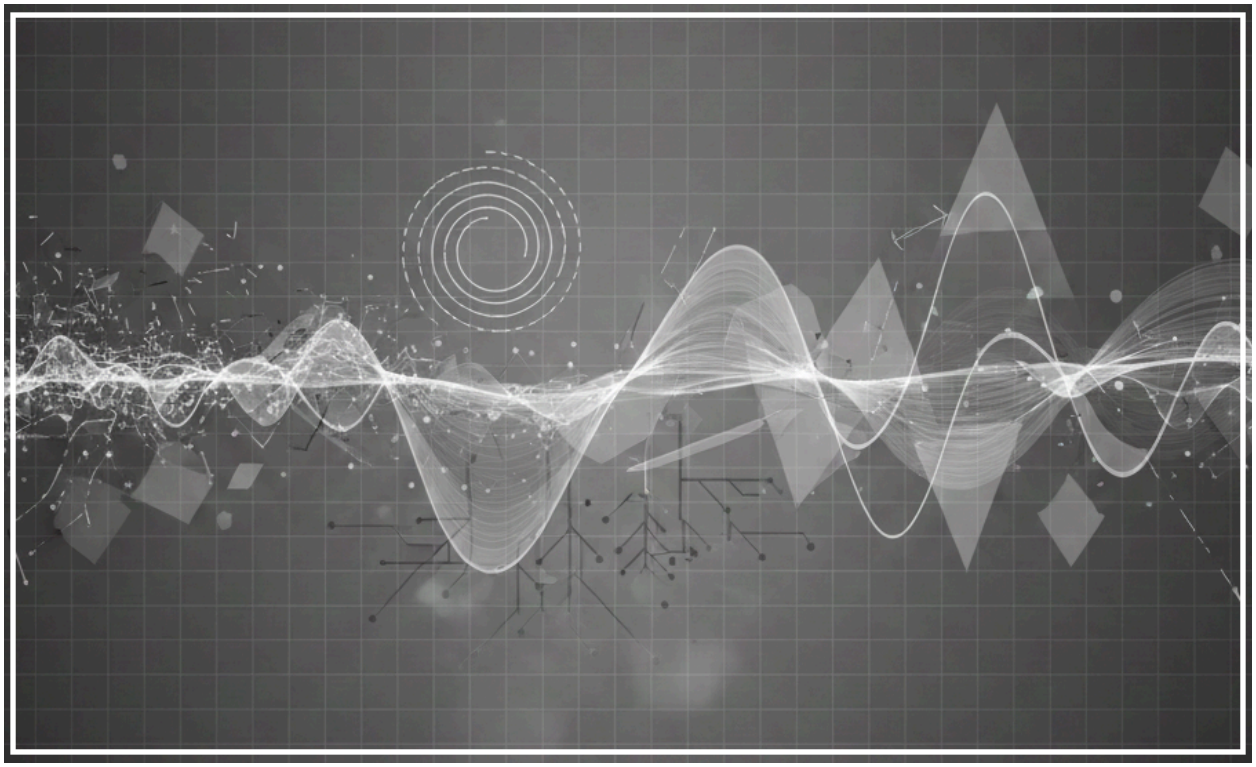
Assignment 1

DATA.STAT.780 Time Series Analysis (Spring 2026, TAU)



Anas Uddin (153215593)

14.02.2026
MSc in Data Science
Column Assigned: V87

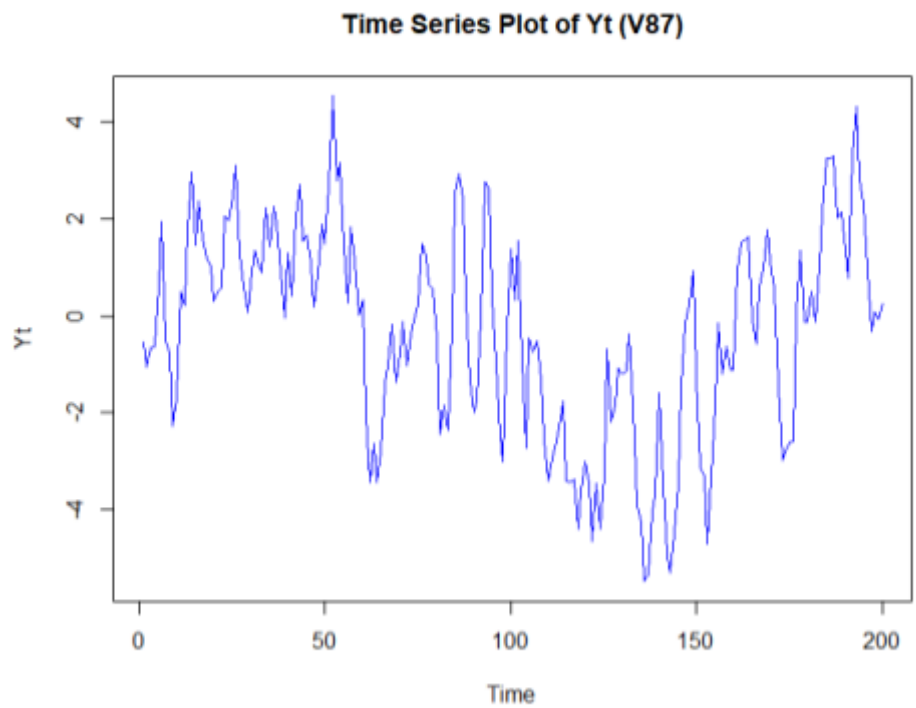
Step 1: Preliminary Analysis of Orders

1. Plot the time series (Y_t)

R Code:

```
# Student Name: Anas Uddin
# Student ID: 153215593
# Assigned Column: V87
# ----------------------------------------

# Step 1: Preliminary analysis of orders ---
# 1. Plot the times series(Yt)

# Load required libraries
library(readr)

# Load the dataset
data <- read_csv("Case_study.csv")

# Extract assigned series
Yt <- data$V87

# Plot the time series
plot(
  Yt,
  type = "l",
  col = "blue",
  main = "Time Series Plot of Yt (V87)",
  xlab = "Time",
  ylab = "Yt"
)
```

Output:



A line plot of the series Y_t over 200 observations

From the time series plot, we can see that the series shows noticeable persistence, and the mean does not appear to be constant over time. There is no clear seasonal pattern. The series does not fluctuate tightly around a fixed mean. This suggests possible non-stationarity, which we confirm next.

2. Analysis of d

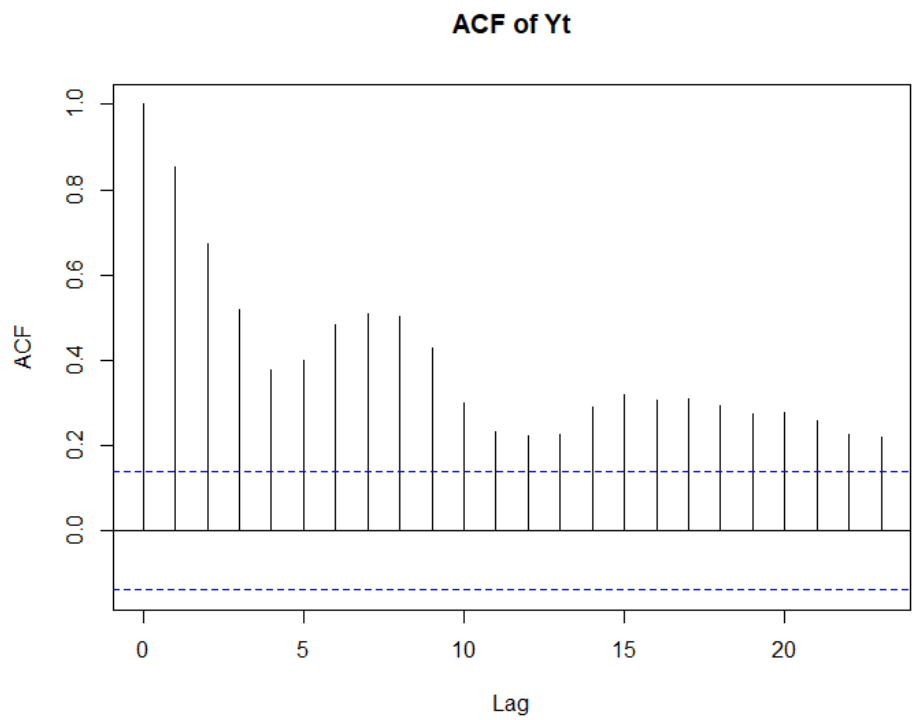
(a) ACF of Y_t, ∇Y_t, and $\nabla^2 Y_t$

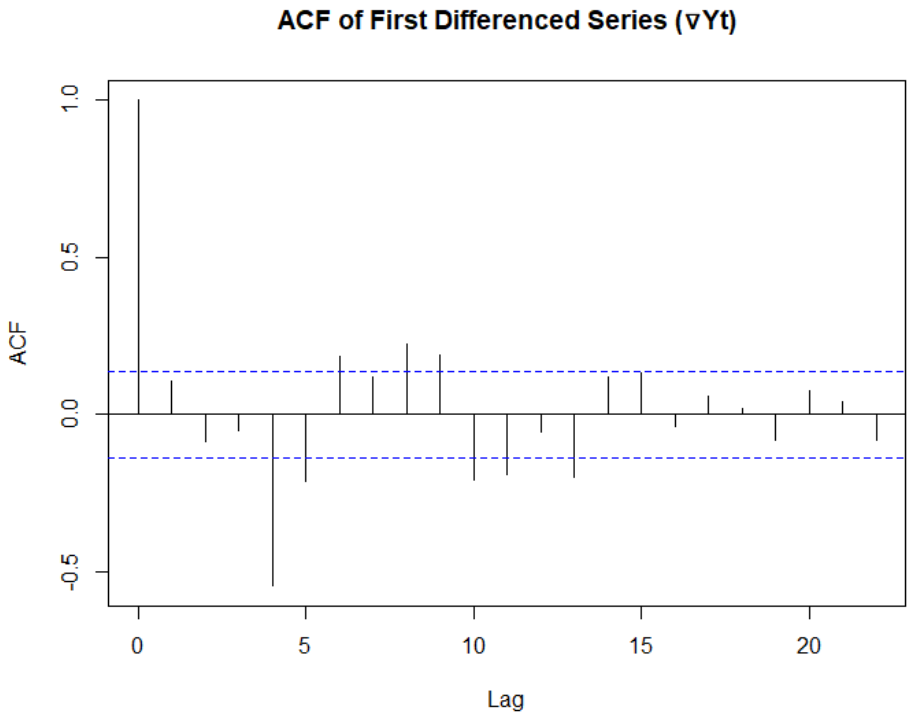
R Code:

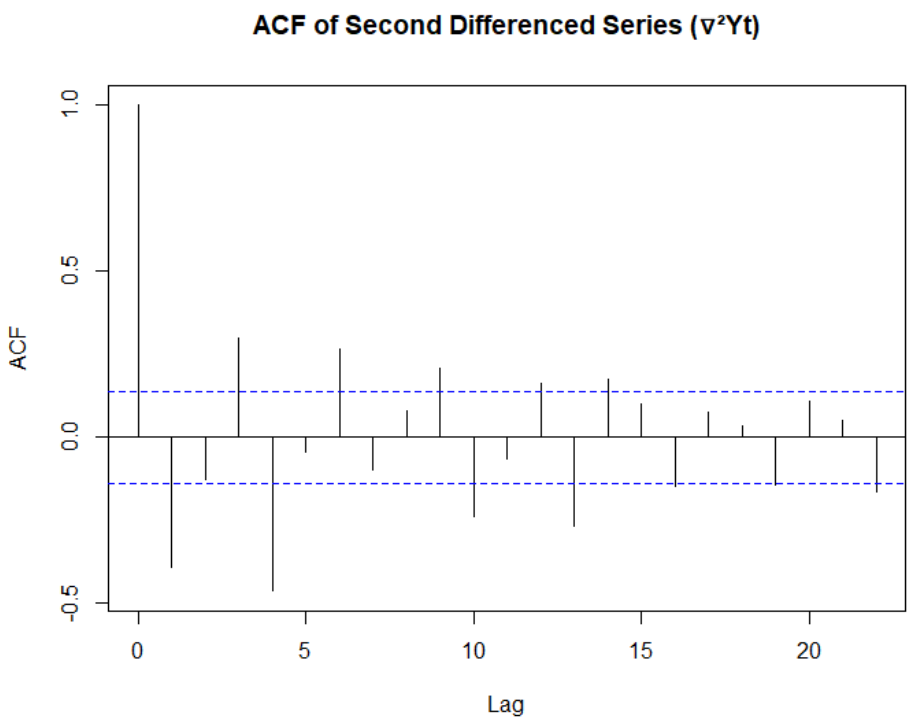
```
# Step 1: Preliminary analysis of orders-- -
# 2. Analysis of d
# (a) ACF of Yt, ∇Yt, and∇²Yt

# ACF of the original series
acf(Yt, main = "ACF of Yt")

# First difference
```

```r
dYt1 <- diff(Yt)
acf(dYt1, main = "ACF of First Differenced Series (∇Yt)")

# Second difference
dYt2 <- diff(dYt1)
acf(dYt2, main = "ACF of Second Differenced Series (∇²Yt)")
```

Output:




ACF of First Differenced Series (∇Yt)
ACF
1.0
0.5
0.0
-0.5
0
5
10
15
20
Lag


ACF of Second Differenced Series (∇²Yt)
ACF
1.0
0.5
0.0
-0.5
0
5
10
15
20
Lag

ACF of Y_t

- The autocorrelations decay very slowly.
- Several lags remain significantly positive.
- The pattern resembles a gradual tailing off.

A slow decay of the ACF is a classic sign of a unit root (non-stationarity).

ACF of ∇Y_t

- Autocorrelations drop off much faster.
- Only the first few lags are significant.
- After that, correlations are small and within bounds.

The first differenced series appears much more stable and likely stationary.

ACF of $\nabla^2 Y_t$

- Autocorrelations are very small.
- Most spikes lie inside confidence bounds.

Second differencing produces a strongly stationary series, possibly over-differenced

(b) Augmented Dickey-Fuller (ADF) test

R Code:

```
# Step 1: Preliminary analysis of orders-- -
# 2. Analysis of d
# (b) Augmented Dickey-Fuller (ADF) test

library(tseries)

# ADF test on original series
adf.test(Yt)

# ADF test on first differenced series
adf.test(dYt1)

# ADF test on second differenced series
adf.test(dYt2)
```

ADF on Y_t

We get from the console output,

Dickey-Fuller = -1.967

Lag order = 5

p-value = 0.5899

The p-value = 0.5899, which is much greater than 0.05. Therefore, fail to reject the null hypothesis of a unit root. So, the original series is non-stationary.

ADF on ∇Y_t

We get from the console output,

Dickey-Fuller = -7.5106

p-value = 0.01

Warning: p-value smaller than printed p-value

The p-value = 0.01, the warning means the true p-value is < 0.01. This is strongly significant. Therefore, reject the null hypothesis of a unit root. So, the first differenced series is stationary.

ADF on $\nabla^2 Y_t$

We get from the console output,

Dickey-Fuller = -13.65

p-value = 0.01

Warning: p-value smaller than printed p-value

We get an extremely small p-value (<< 0.01). Strong rejection of the unit root. However, second differencing is unnecessary because stationarity was already achieved at the first difference.

(c) Suggested order for d

d = 1, The Augmented Dickey-Fuller test fails to reject the unit root hypothesis for the original series but strongly rejects it for the first differenced series. Therefore, the series

is integrated of order one, and further differencing is unnecessary.

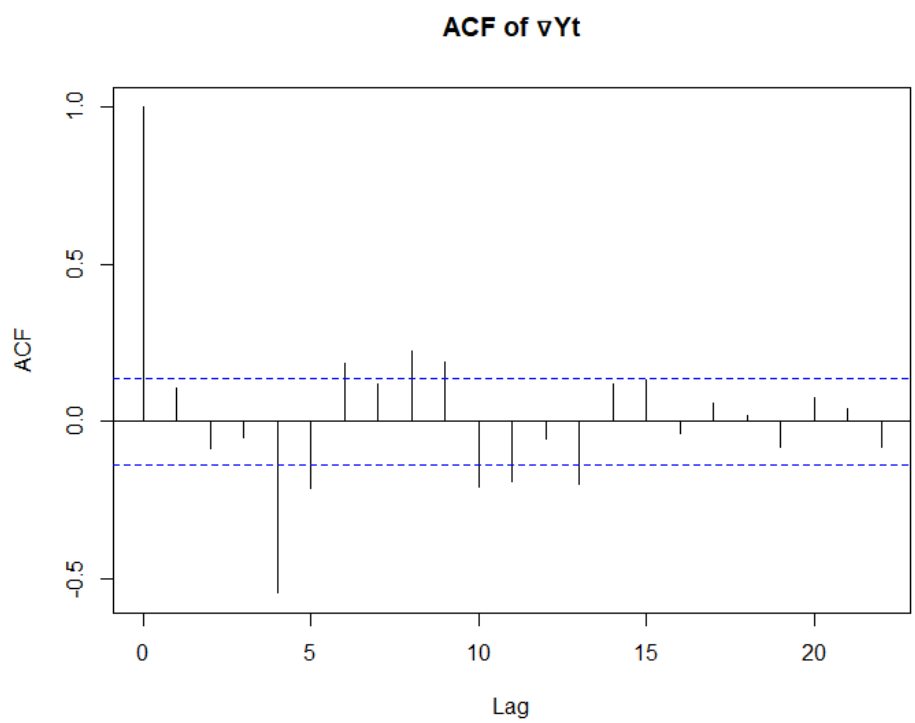
3. Analysis of autoregressive order p and moving average order q

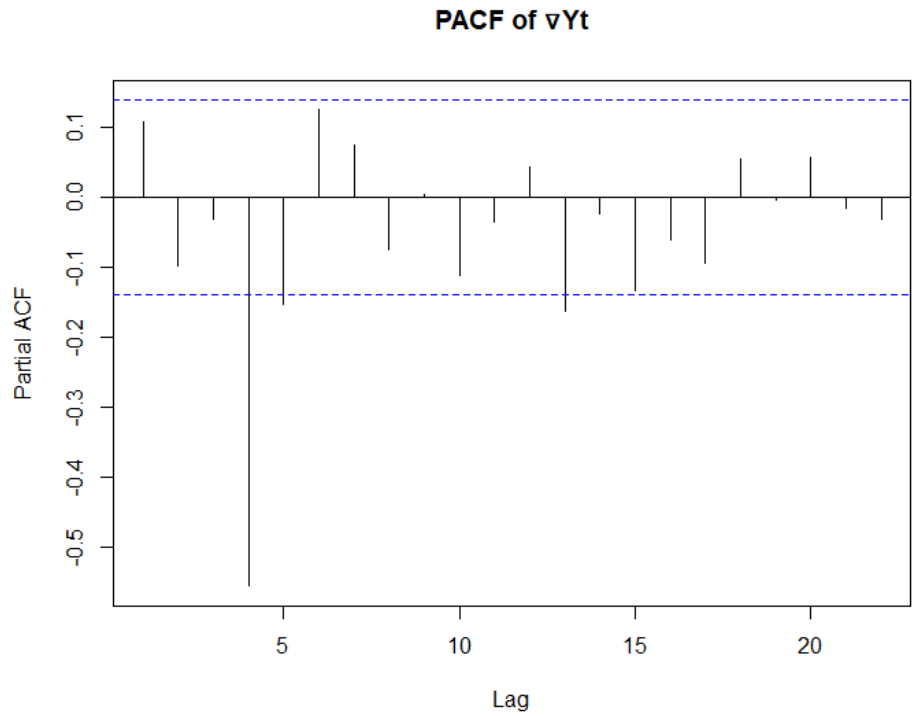
Using the stationary series ∇Y_t

R Code:

```
# Step 1: Preliminary analysis of orders-- -
# 3. Analysis of autoregressive order p and moving average order q

# ACF and PACF of first differenced series
acf(dYt1, main = "ACF of ∇Yt")
pacf(dYt1, main = "PACF of ∇Yt")
```

Output:





ACF of ∇Y_t

Significant spikes in the first few lags. After lag 3-4, most values lie within confidence bounds. This suggests $q_{max} \approx 3$ or 4

PACF of ∇Y_t

Significant spikes in early lags. PACF also tapers after about lag 3-4. This suggests qmax ≈ 3 or 4

So, reasonable upper bounds are:

$$p_{max} = 4, q_{max} = 4$$

These bounds will guide the ARIMA model estimation and selection in the next step.

Step 2: Estimation and Selection of ARIMA Models

1. Model estimation and information criteria

We estimate ARIMA(p,1,q) for: $1 \leq p \leq 4$, $1 \leq q \leq 4$

This gives 16 candidate models. We use the first 100 observations as the in-sample period.

R Code:

```
# Step 2: Estimation and selection of ARIMA Models ---
# 1. Model estimation and information criteria

library(readr)

# Load data
data <- read_csv("Case_study.csv")

# Extract assigned series
Yt <- data$V87

# Use first 100 observations as training data
Y_train <- Yt[1:100]

# Differencing order (from Step 1)
d <- 1

# Maximum orders
pmax <- 4
qmax <- 4

# Create empty storage
results <- data.frame(
  p = integer(),
  q = integer(),
  AIC = numeric(),
  BIC = numeric()
)

# Loop over all combinations 1 ≤ p ≤ 4, 1 ≤ q ≤ 4
for (p in 1:pmax) {
```

```
  for (q in 1:qmax) {

    model <- arima(Y_train, order = c(p, d, q))

    results <- rbind(results,
                     data.frame(
                       p = p,
                       q = q,
                       AIC = AIC(model),
                       BIC = BIC(model)
                     ))
  }
}

# Display results
results
```

Output:

The following table contains AIC and BIC for all 16 ARIMA(p,1,q) models.

```
   p q      AIC      BIC
1  1 1 314.6911 322.4765
2  1 2 305.5215 315.9020
3  1 3 307.3737 320.3493
4  1 4 285.8333 301.4040
5  2 1 316.3768 326.7572
6  2 2 305.5268 318.5024
7  2 3 288.4406 304.0113
8  2 4 286.8699 305.0357
9  3 1 303.3413 316.3169
10 3 2 306.4885 322.0592
11 3 3 300.4716 318.6374
12 3 4 285.2781 306.0390
13 4 1 273.1528 288.7235
14 4 2 273.9592 292.1250
15 4 3 275.8691 296.6300
16 4 4 277.1603 300.5164
```

The minimum AIC is: ARIMA(4,1,1). The minimum BIC is also: ARIMA(4,1,1). Both AIC and BIC select the same model. This strengthens the selection because AIC favors predictive efficiency. BIC penalizes complexity more strongly. When both agree, the

model is strongly supported.

2. Selection of the best three specifications

We now identify the minimum AIC model. Identify the minimum BIC model. Include adjacent models with information criteria close to the minimum.

R Code:

```
# Step 2: Estimation and selection of ARIMA Models ---
# 2. Selection of the best three specifications

# Order by AIC
results_AIC <- results[order(results$AIC), ]

# Order by BIC
results_BIC <- results[order(results$BIC), ]

# Best three according to AIC
best_AIC_models <- head(results_AIC, 3)

# Best three according to BIC
best_BIC_models <- head(results_BIC, 3)

best_AIC_models
best_BIC_models
```

Output:

The following two tables show the three best models according to AIC and BIC.

```
> best_AIC_models
   p q      AIC      BIC
13 4 1 273.1528 288.7235
14 4 2 273.9592 292.1250
15 4 3 275.8691 296.6300
> best_BIC_models
   p q      AIC      BIC
13 4 1 273.1528 288.7235
14 4 2 273.9592 292.1250
15 4 3 275.8691 296.6300
```

The three best models (according to both AIC and BIC) are:

ARIMA(4,1,1)

ARIMA(4,1,2)

ARIMA(4,1,3)

However, the AIC difference between (4,1,1) and (4,1,2) is: 273.9592 − 273.1528 = 0.8064. This is very small.

The AIC difference between (4,1,1) and (4,1,3) is: 275.8691 − 273.1528 = 2.7163. This is larger but still moderate.

According to model selection practice, differences < 2 denote models are very similar. Differences > 4 denote clearly worse.

So, ARIMA(4,1,2) is a serious competitor. ARIMA(4,1,3) is less competitive.

3. Parameter estimates of the best three specifications

We now estimate the three selected models and show their coefficients.

R Code:

```
# Step 2: Estimation and selection of ARIMA Models ---
# 3. Parameter estimates of the best three specifications

# Select three best unique models (based on AIC and BIC)
best_models <- unique(rbind(best_AIC_models[1:3, c("p","q")],
                            best_BIC_models[1:3, c("p","q")]))

# Fit and display each model
for (i in 1:nrow(best_models)) {

  p_val <- best_models$p[i]
  q_val <- best_models$q[i]

  cat("\n==============================\n")
  cat("ARIMA(", p_val, ",1,", q_val, ")\n", sep="")

  model <- arima(Y_train, order=c(p_val,1,q_val))
  print(model)
}
```

Output:

From the output we got,

ARIMA(4,1,1) (Best Model)

Estimates:

Parameter	Estimate	Std. Error
ar1	0.1969	0.1110
ar2	-0.1094	0.0829
ar3	0.0001	0.0812
ar4	-0.6321	0.0799
ma1	-0.1937	0.1341

We can see that ar4 = -0.6321 is large relative to its standard error, which is clearly significant. ar1 is moderately sized. ar3 ≈ 0 (very small, likely insignificant). ma1 is moderate.

Model fit: $\sigma^2 = 0.8001$. log likelihood = -130.58. AIC = 273.15 (lowest)

The fourth autoregressive term plays a major role in explaining the dynamics. This indicates dependence up to lag 4 in the differenced series.

ARIMA(4,1,2)

Shows slightly lower variance (0.7895), slightly higher AIC (273.96), and adds an additional MA(2) parameter. However, the improvement in log-likelihood is very small. BIC increases noticeably (292.13 vs 288.72). So, the added MA(2) term does not improve the model sufficiently to justify the extra complexity.

ARIMA(4,1,3)

We can see that AIC increases further (275.87), and BIC increases substantially (296.63). Several parameters have relatively large standard errors. So, this model appears over-parameterized.

Finally, based on AIC and BIC, the preferred model is ARIMA(4,1,1)

These three specifications will be further evaluated using diagnostic tests in Step 3.

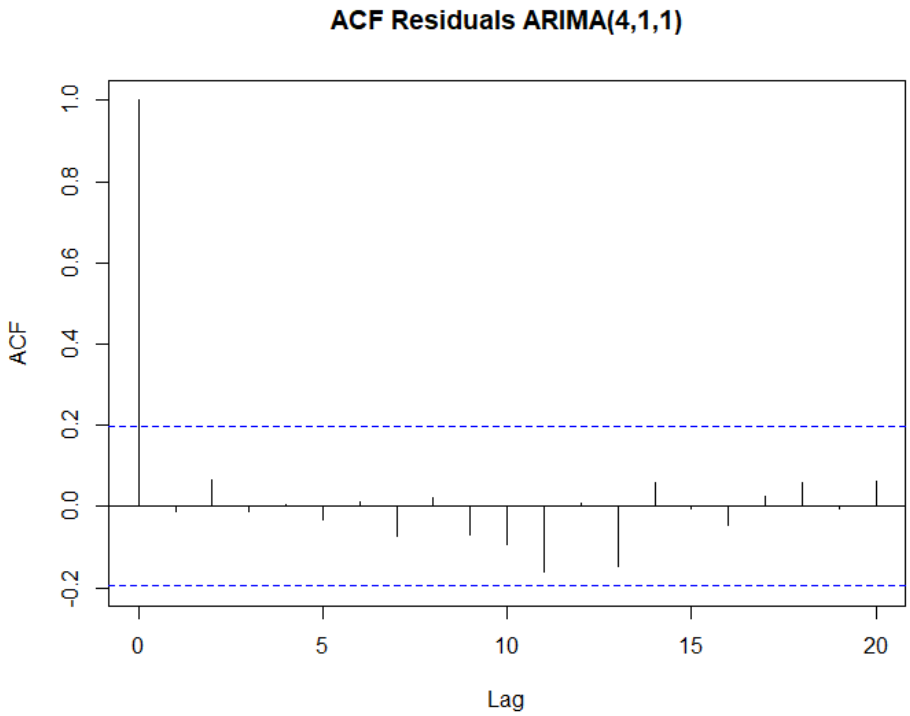
Step 3: Diagnostic Tests with In-Sample Data

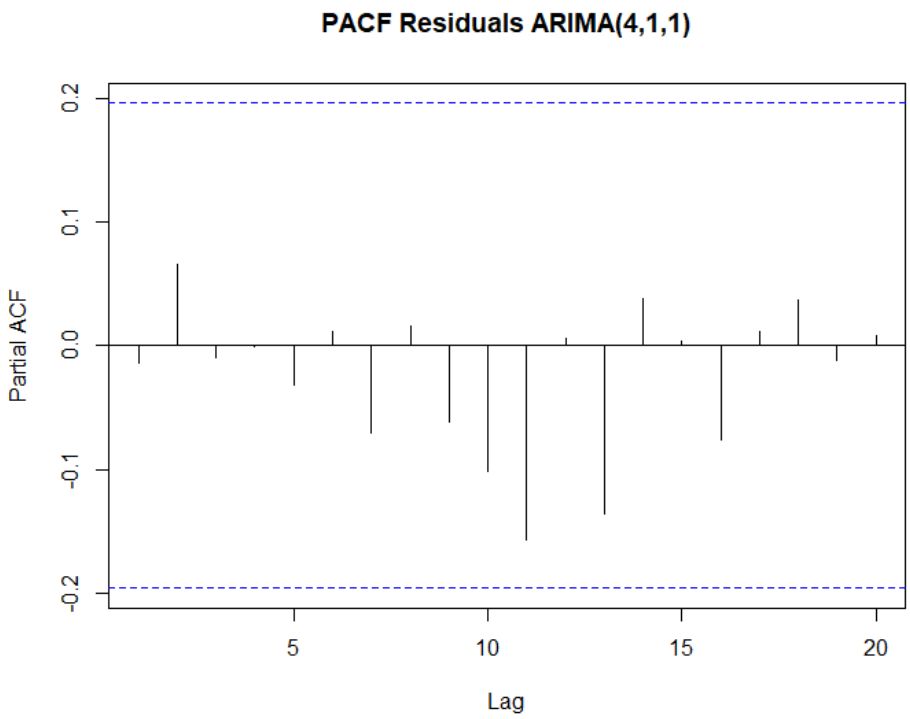
1. LjungBox test (first 10 lags)

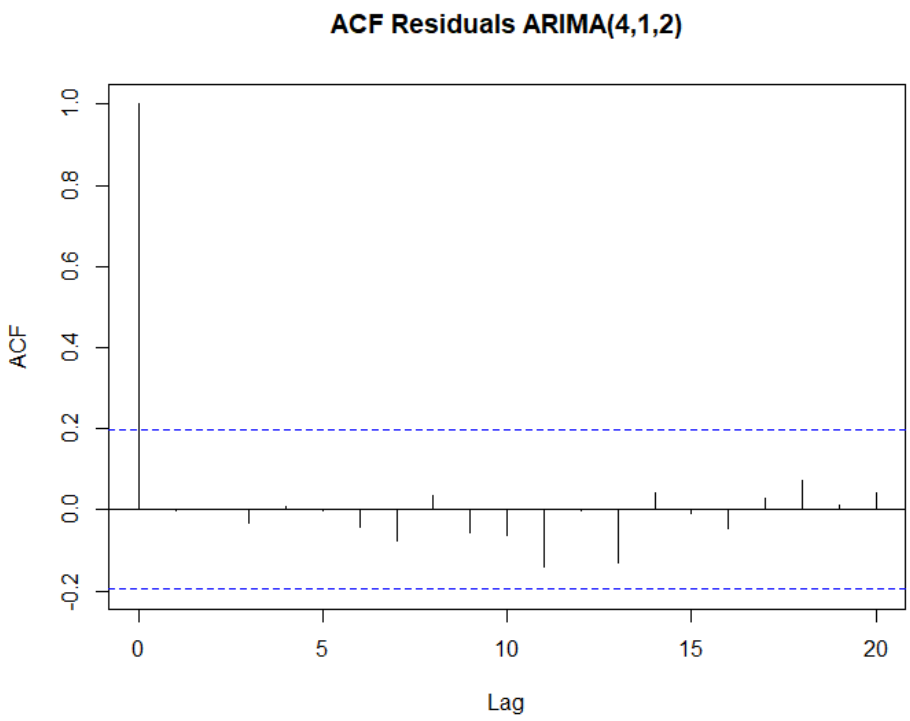
R Code:

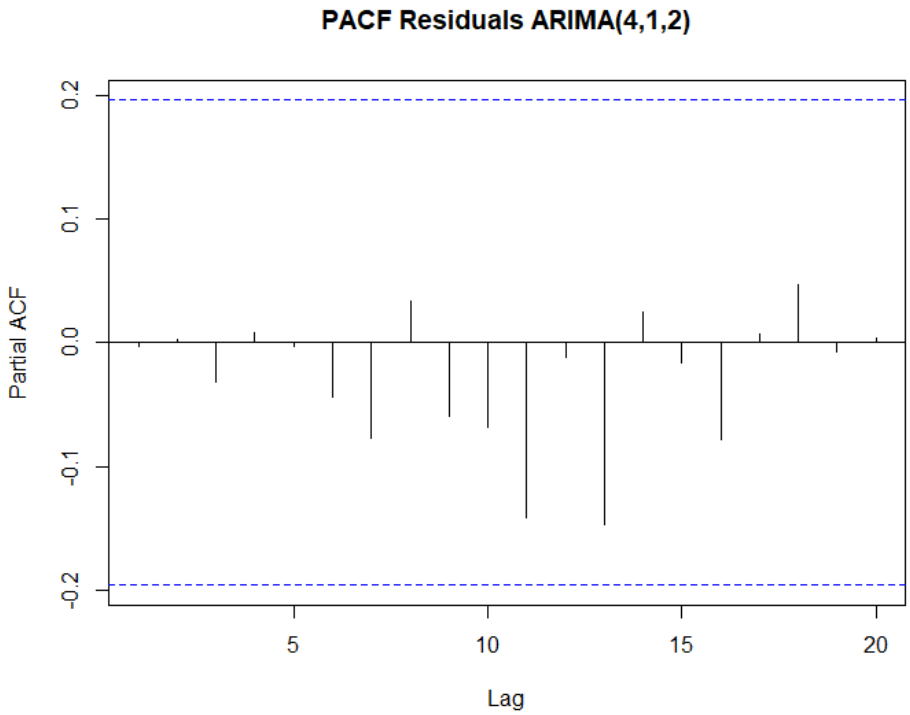
```
# Step 3: Diagnostic Tests with In-Sample Data ---
# 1. LjungBox test (first 10 lags)

# Fit the three selected models again
model_411 <- arima(Y_train, order=c(4,1,1))
model_412 <- arima(Y_train, order=c(4,1,2))
model_413 <- arima(Y_train, order=c(4,1,3))

# Extract residuals
res_411 <- residuals(model_411)
res_412 <- residuals(model_412)
res_413 <- residuals(model_413)

# Ljung-Box test (lag = 10)
Box.test(res_411, lag=10, type="Ljung-Box")
Box.test(res_412, lag=10, type="Ljung-Box")
Box.test(res_413, lag=10, type="Ljung-Box")

# ACF and PACF of residuals
acf(res_411, main="ACF Residuals ARIMA(4,1,1)")
pacf(res_411, main="PACF Residuals ARIMA(4,1,1)")

acf(res_412, main="ACF Residuals ARIMA(4,1,2)")
pacf(res_412, main="PACF Residuals ARIMA(4,1,2)")

acf(res_413, main="ACF Residuals ARIMA(4,1,3)")
pacf(res_413, main="PACF Residuals ARIMA(4,1,3)")
```

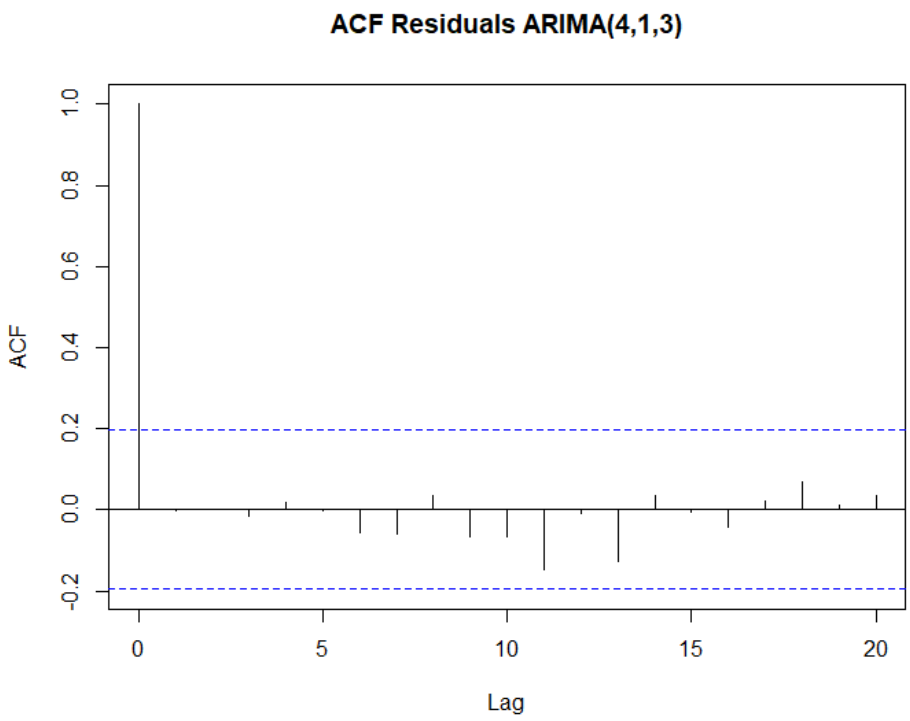
Output:

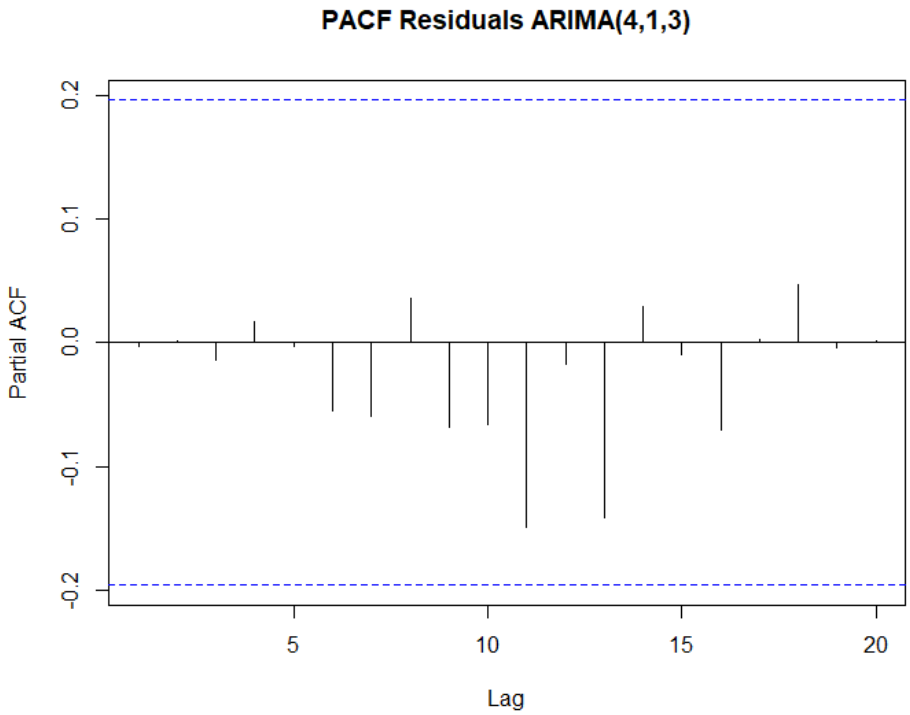

ACF Residuals ARIMA(4,1,1)
ACF
1.0
0.8
0.6
0.4
0.2
0.0
-0.2
0
5
10
15
20
Lag


PACF Residuals ARIMA(4,1,1)
Partial ACF
0.2
0.1
0.0
-0.1
-0.2
5
10
15
20
Lag


ACF Residuals ARIMA(4,1,2)
ACF
1.0
0.8
0.6
0.4
0.2
0.0
-0.2
0
5
10
15
20
Lag


PACF Residuals ARIMA(4,1,2)
Partial ACF
0.2
0.1
0.0
-0.1
-0.2
5
10
15
20
Lag


ACF Residuals ARIMA(4,1,3)
ACF
1.0
0.8
0.6
0.4
0.2
0.0
-0.2
0
5
10
15
20
Lag


PACF Residuals ARIMA(4,1,3)
Partial ACF
0.2
0.1
0.0
-0.1
-0.2
5
10
15
20
Lag

From the output we got,

Model	Test Statistic	p-value
ARIMA(4,1,1)	2.8323	0.9851
ARIMA(4,1,2)	1.8918	0.9971
ARIMA(4,1,3)	1.8953	0.9971

The LjungBox test examines whether residual autocorrelations up to lag 10 are jointly zero.

For all three specifications, the p-values are extremely large (all above 0.98). Since all p-values are much greater than 0.05, we fail to reject the null hypothesis of no autocorrelation.

This implies there is no evidence of serial correlation in the residuals. The residuals behave like white noise. All three ARIMA specifications adequately capture the dynamic structure of the series.

From the residual ACF and PACF plots, we can see that all autocorrelation spikes lie within the 95% confidence bounds. No systematic pattern or significant lag structure is visible. This visually confirms the LjungBox test results: the residuals appear uncorrelated.

2. Normality of residuals

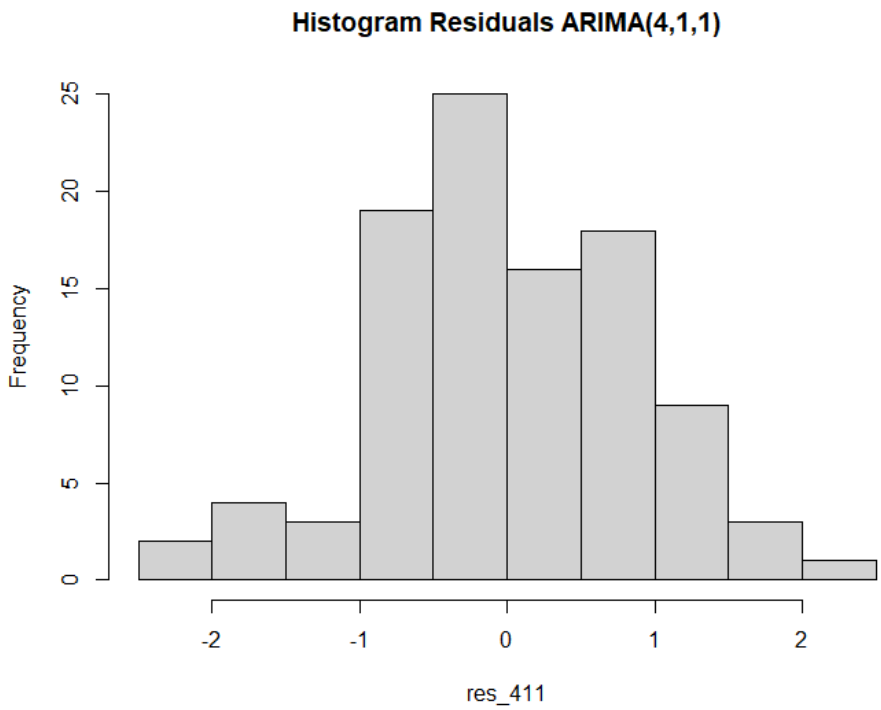
We test, H_0 : Residuals are normally distributed

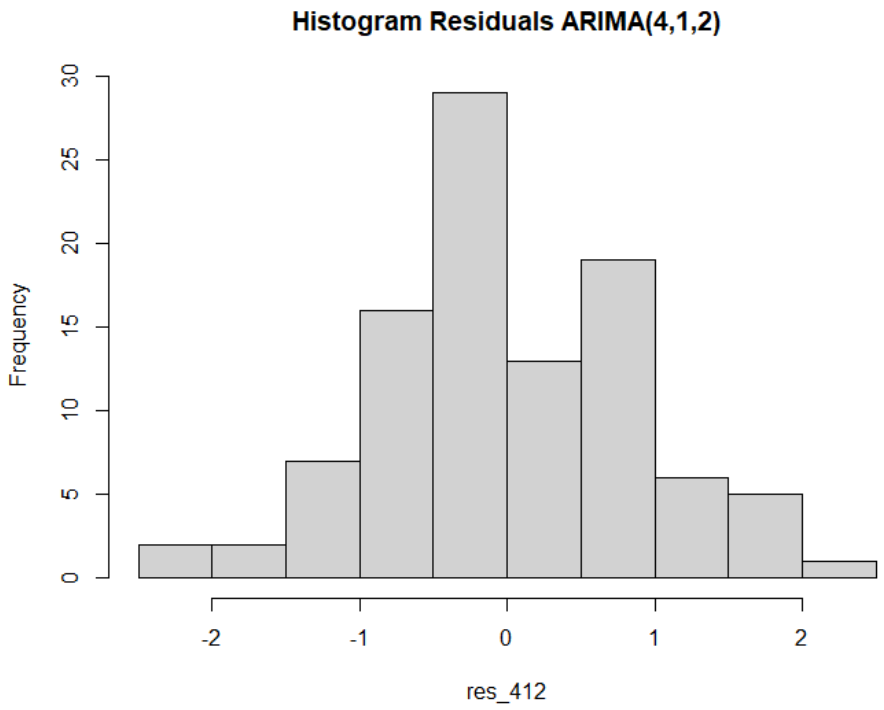
R Code:

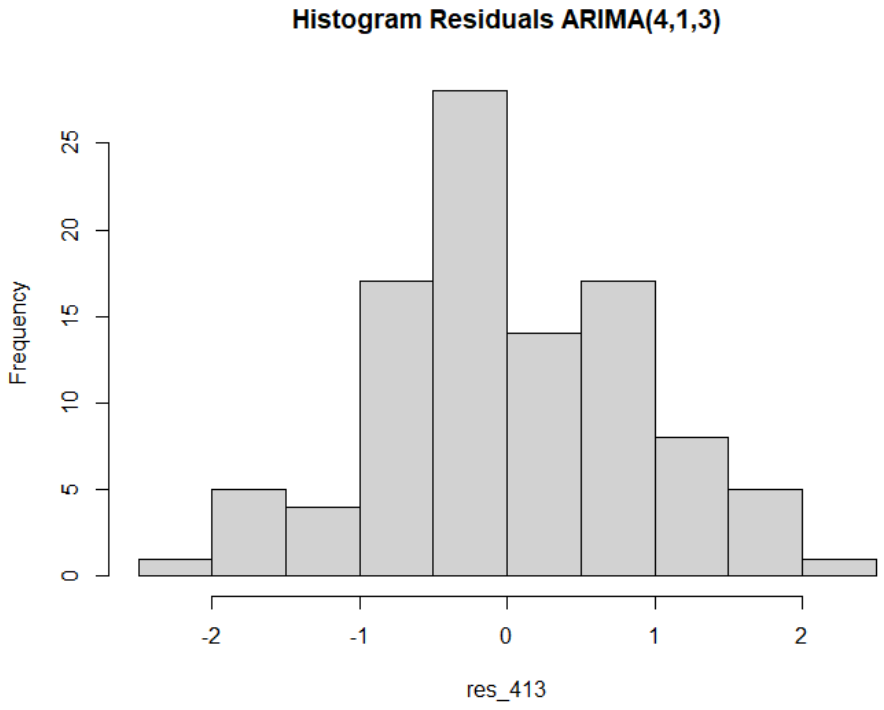
```
# Step 3: Diagnostic Tests with In-Sample Data ---
# 2. Normality of residuals

# Histograms
hist(res_411, main="Histogram Residuals ARIMA(4,1,1)")
hist(res_412, main="Histogram Residuals ARIMA(4,1,2)")
hist(res_413, main="Histogram Residuals ARIMA(4,1,3)")

# QQ plots
qqnorm(res_411); qqline(res_411)
qqnorm(res_412); qqline(res_412)
qqnorm(res_413); qqline(res_413)
```

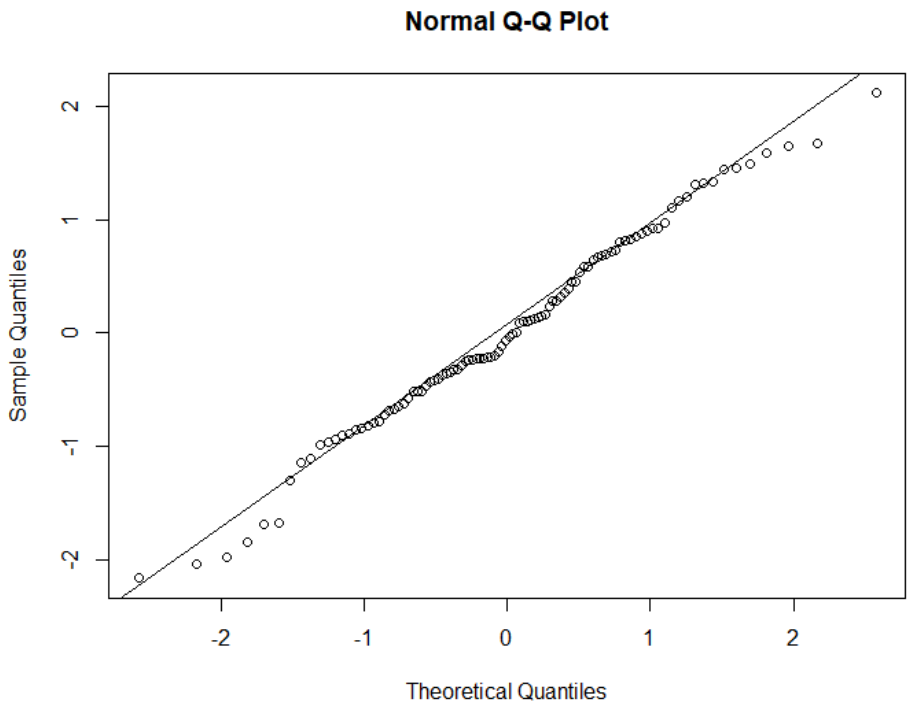
```
# Shapiro-Wilk test
shapiro.test(res_411)
shapiro.test(res_412)
shapiro.test(res_413)
```

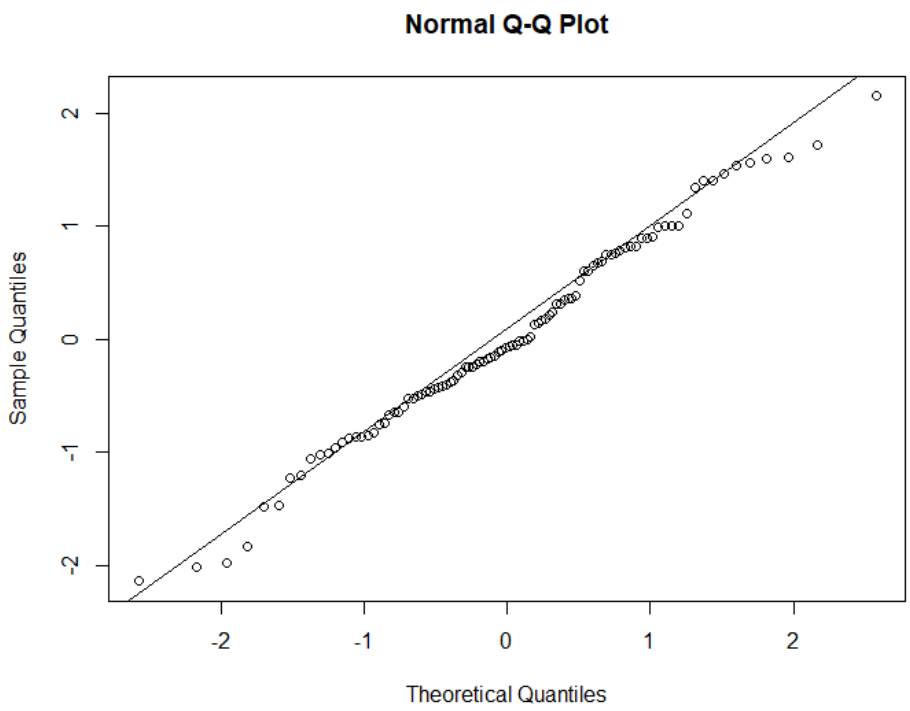
Output:

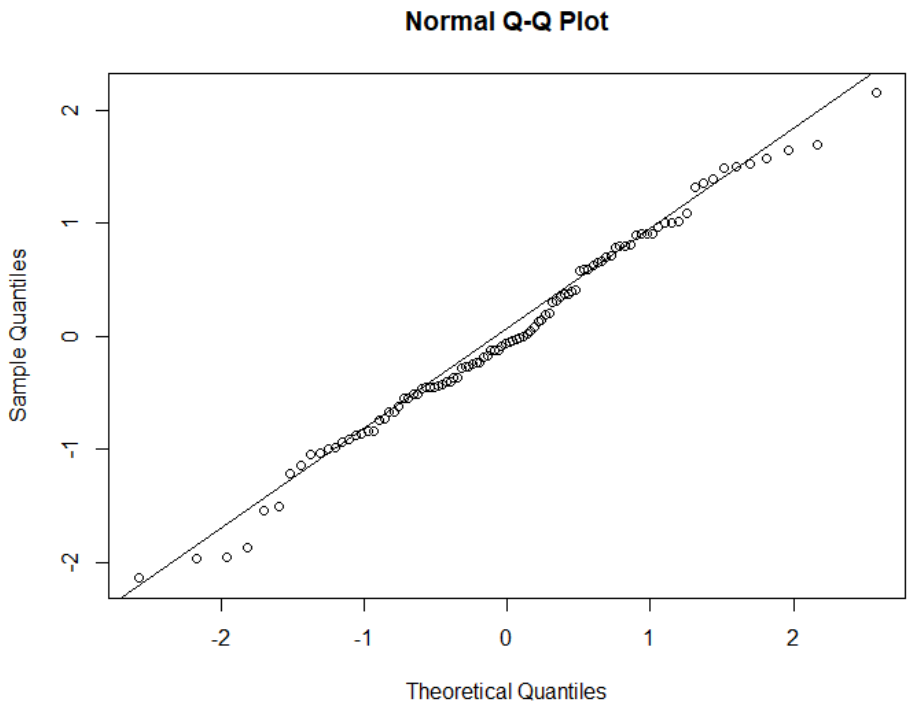



Histogram Residuals ARIMA(4,1,2)
Frequency
0
5
10
15
20
25
30
-2
-1
0
1
2
res_412


Histogram Residuals ARIMA(4,1,3)
Frequency
0
5
10
15
20
25
-2
-1
0
1
2
res_413


Normal Q-Q Plot
Sample Quantiles
Theoretical Quantiles
-2
-1
0
1
2


Normal Q-Q Plot
Sample Quantiles
Theoretical Quantiles
-2
-1
0
1
2



Shapiro-Wilk test results,

Model	W Statistic	p-value
ARIMA(4,1,1)	0.98871	0.5621
ARIMA(4,1,2)	0.98923	0.6030
ARIMA(4,1,3)	0.98961	0.6331

The null hypothesis of the Shapiro-Wilk test is that the residuals are normally distributed.

For all three models, the p-values are well above 0.05. Therefore, we fail to reject the null hypothesis of normality.

The histograms appear approximately symmetric and bell-shaped. The QQ-plots show residual points closely aligned with the reference line. No strong deviations in the tails are visible.

Hence, the residuals of all three models can be considered approximately normally distributed.

3. Preferred Model

From the Step 2 and Step 3 results, all three models pass autocorrelation tests. All three models pass normality tests. There is no diagnostic problem in any specification.

According to the principle of parsimony, when two competing models perform similarly well, the model with fewer parameters is preferred.

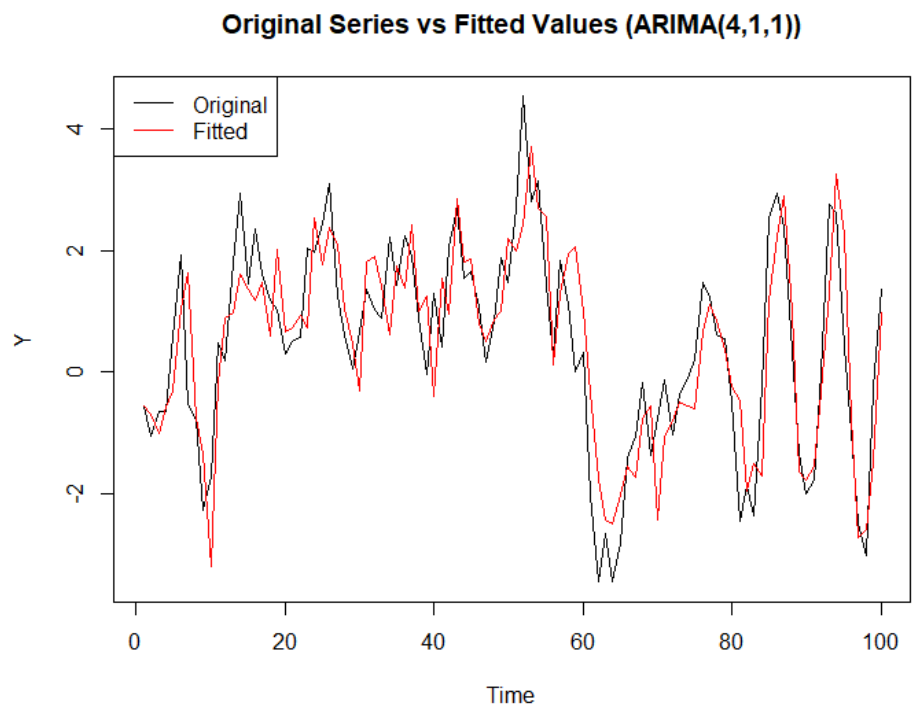
Since ARIMA(4,1,1) has fewer moving average parameters than ARIMA(4,1,2) and ARIMA(4,1,3), comparable (and acceptable) diagnostic performance, ARIMA(4,1,1) is selected as the preferred specification.

4. Plot original series and fitted values

R Code:

```
# Step 3: Diagnostic Tests with In-Sample Data ---
# 4. Plot original series and fitted values

# Extract fitted values
fitted_411 <- Y_train - res_411

# Plot
plot(Y_train, type="l", col="black",
     main="Original Series vs Fitted Values (ARIMA(4,1,1))",
     xlab="Time",
     ylab="Y")

lines(fitted_411, col="red")
legend("topleft", legend=c("Original","Fitted"),
       col=c("black","red"), lty=1)
```

Output:



From the plot of the original series (black line)and fitted values from ARIMA(4,1,1) (red line), we observe that the fitted values closely track the movements of the original series. No systematic deviation or pattern is visible. The model captures the short-run dynamics well.

This further supports the adequacy of the ARIMA(4,1,1) specification.

Step 4: Forecast with Out-of-Sample Data

1. Generate forecasts (h = 10, 25, 100)

R Code:

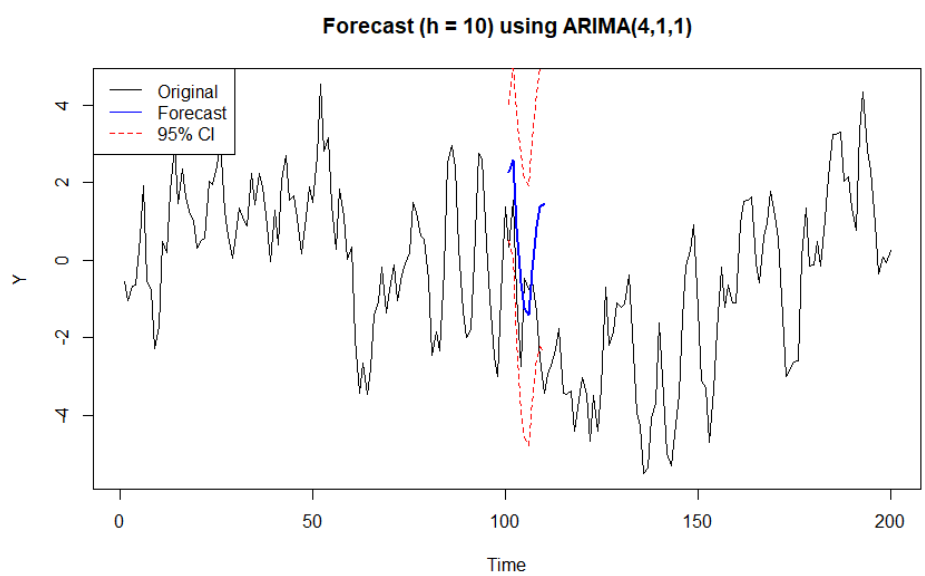
```
# Step 4: Forecast with Out-of-Sample Data ---
# 1. Generate forecasts (h = 10, 25, 100)

# Preferred model from Step 3
final_model <- arima(Y_train, order = c(4, 1, 1))
```

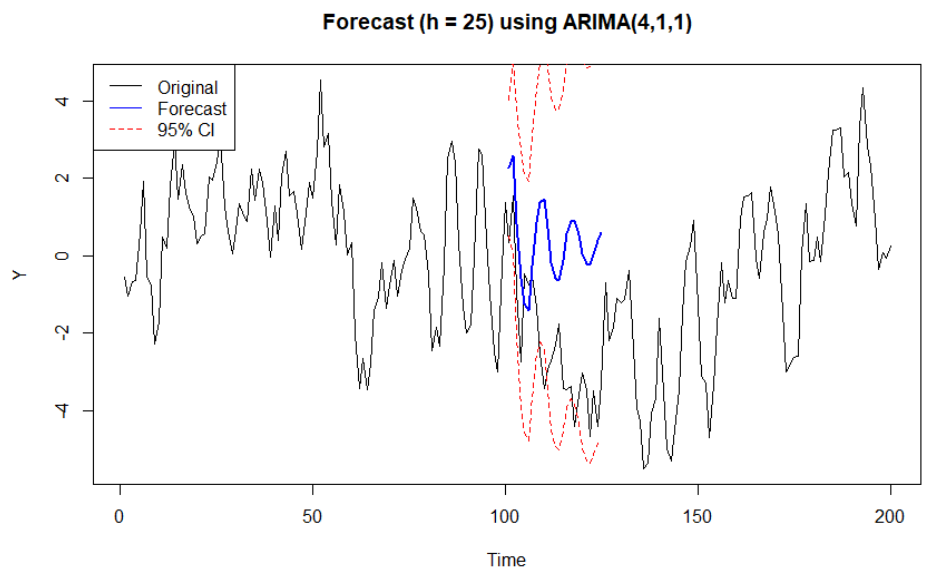
```r
# Forecast horizons
forecast_10  <- predict(final_model, n.ahead = 10)
forecast_25  <- predict(final_model, n.ahead = 25)
forecast_100 <- predict(final_model, n.ahead = 100)

# Extract forecast means and standard errors
pred_10  <- forecast_10$pred
se_10    <- forecast_10$se

pred_25  <- forecast_25$pred
se_25    <- forecast_25$se

pred_100 <- forecast_100$pred
se_100   <- forecast_100$se

# 95% Confidence Intervals
upper_10  <- pred_10  + 1.96 * se_10
lower_10  <- pred_10  - 1.96 * se_10

upper_25  <- pred_25  + 1.96 * se_25
lower_25  <- pred_25  - 1.96 * se_25

upper_100 <- pred_100 + 1.96 * se_100
lower_100 <- pred_100 - 1.96 * se_100

# Forecast plot (h = 10)
plot(
  Y,
  type = "l",
  col = "black",
  main = "Forecast (h = 10) using ARIMA(4,1,1)",
  xlab = "Time",
  ylab = "Y"
)

# Add forecast line
lines(101:110, pred_10, col = "blue", lwd = 2)

# Add 95% confidence interval
lines(101:110, upper_10, col = "red", lty = 2)
lines(101:110, lower_10, col = "red", lty = 2)
```

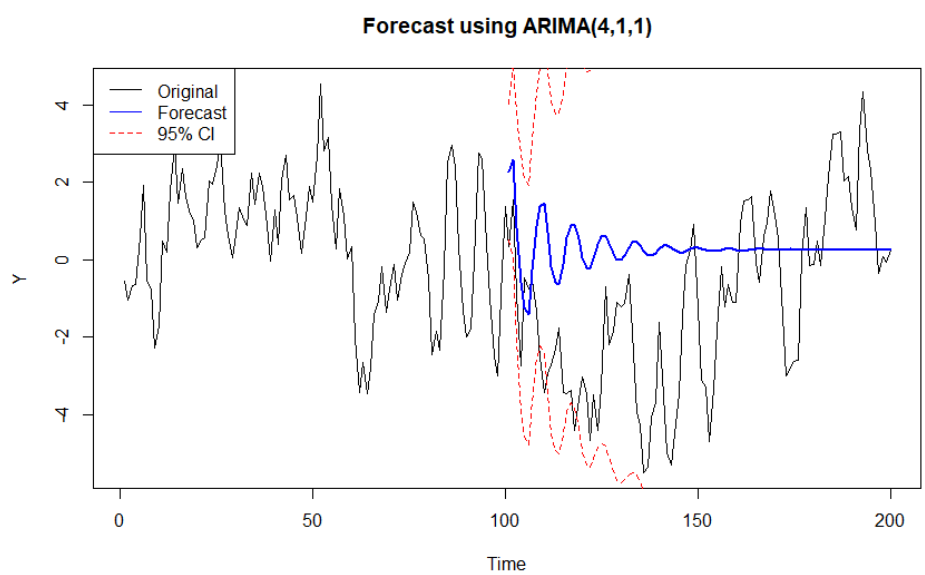
```r
legend(
  "topleft",
  legend = c("Original", "Forecast", "95% CI"),
  col = c("black", "blue", "red"),
  lty = c(1, 1, 2)
)

# Forecast plot (h = 25)
plot(
  Y,
  type = "l",
  col = "black",
  main = "Forecast (h = 25) using ARIMA(4,1,1)",
  xlab = "Time",
  ylab = "Y"
)

# Add forecast line
lines(101:125, pred_25, col = "blue", lwd = 2)

# Add 95% confidence interval
lines(101:125, upper_25, col = "red", lty = 2)
lines(101:125, lower_25, col = "red", lty = 2)

legend(
  "topleft",
  legend = c("Original", "Forecast", "95% CI"),
  col = c("black", "blue", "red"),
  lty = c(1, 1, 2)
)

# Forecast plot (h = 100)
# Plot original series (all 200 observations)
plot(
  Y,
  type = "l",
  col = "black",
  main = "Forecast using ARIMA(4,1,1)",
  xlab = "Time",
  ylab = "Y"
)
```

```r
# Add forecast (starting at 101)
lines(101:200, pred_100, col = "blue", lwd = 2)

# Add confidence intervals
lines(101:200, upper_100, col = "red", lty = 2)
lines(101:200, lower_100, col = "red", lty = 2)

legend(
  "topleft",
  legend = c("Original", "Forecast", "95% CI"),
  col = c("black", "blue", "red"),
  lty = c(1, 1, 2)
)
```

Output:







Using the preferred specification ARIMA(4,1,1), forecasts were generated for:

Short horizon: h=10

Medium horizon: h=25

Long horizon: h=100

Each forecast includes a 95% confidence interval.

h = 10 (Short-term forecast)

- The forecast closely follows the recent movement of the series.
- The confidence interval is relatively narrow.
- This indicates high precision in short-term predictions.
- Short-run dynamics are well captured by the ARIMA(4,1,1) model.

h = 25 (Medium-term forecast)

- The forecast becomes smoother compared to h = 10.
- The confidence interval widens.
- Forecast uncertainty increases as the horizon expands.
- The model begins to rely more on its long-run structure rather than recent fluctuations.

h = 100 (Long-term forecast)

- The forecast path becomes considerably smoother.
- The confidence bands widen substantially.
- This reflects increasing uncertainty for long-horizon predictions.
- The model gradually converges toward its long-run stochastic behavior.

This pattern is fully consistent with ARIMA forecasting theory: Forecast variance increases with the forecast horizon.

2. Mean Squared Error (MSE)

MSE is calculated using the out-of-sample data (observations 101-200).

R Code:

```
# Step 4: Forecast with Out-of-Sample Data ---
# 2. Mean Squared Error (MSE)

# Split the data
Y_test  <- Y[101:200]

# Compute forecast errors for 100-step forecast
errors <- Y_test - pred_100

# Mean Squared Error
MSE <- mean(errors^2)

MSE
```

Output:

Using the 100-step ahead forecast and the actual out-of-sample observations (periods 101-200), the Mean Squared Error is: 7.420053

The MSE measures the average squared difference between actual and forecasted values. An MSE of 7.420053 indicates the average magnitude of forecast error over the 100 out-of-sample observations. Since the model passed all diagnostic tests in Step 3 and was selected based on AIC, BIC, and parsimony, this MSE represents the genuine out-of-sample forecasting performance of the preferred specification.

So, ARIMA(4,1,1) remains the final preferred forecasting model. The model provides stable and reasonable predictive performance for my assigned series V87.